

Can BeeHive run a 300-attendee conference?

A read-only audit of the live implementation — LiveKit configuration, media defaults, per-join backend work, the presence layer, and one measured mass-join burst. Findings are traced to file and line; nothing was changed.

audited build 1.0.294 · status updated for 1.0.303 branch staging

livekit-client 2.20 · components-react 2.9.21 2026-09-11 · updated 2026-09-16

● RED — CONTAINED

Not as-is. **Conditional** after four contained fixes — none of them in the media path.

The LiveKit core is sound for 300 in audience mode. The blockers live in BeeHive's own presence and realtime layer, plus one waiting-room gap. This is two subsystems, not a rebuild.

PRACTICAL MAXIMUM TODAY

~50 comfortable

~100 at the edge

After P0 + P1: 300 in audience mode, with ≤ 8 cameras and ≤ 20 unmuted at any moment.

STATUS UPDATE — BUILD 1.0.303 (16 SEP 2026)

Every code-side blocker in this audit has since shipped. What remains is infrastructure confirmation, which no code change can substitute for.

- **P0 #1 presence** — roster now derived from LiveKit participant attributes; the `N2 room_participants` refetch is gone (v1.0.298).
- **P0 #2 fan-out** — cursors, reactions, hands and moderation signals ride LiveKit data channels, not Supabase broadcast (v1.0.298).
- **P0 #3 bulk admit** — `/admit { all: true }` + "Admit all" in the waiting-room panel (v1.0.298).
- **P1 adaptiveStream / dynacast** — enabled, plus a hard cap of 20 live microphones with one-click ask-to-unmute (v1.0.299).
- **P1 audience role** — "Large session" rooms issue `canPublish:false` tokens; host/co-host promote speakers one at a time (v1.0.301).

- **P1 participants panel** — windowed above 60 participants, tiles memoized; verified with a 301-person roster (v1.0.303).
- **Ops Dashboard** — admin view of plan readiness, room and mic counts (v1.0.300).

Still open — P0 #4: confirm Supabase plan (Realtime peak connections $\geq$ 300 $\rightarrow$ Pro or above) and the LiveKit Cloud quota, then set `SUPABASE_PLAN` and `LIVEKIT_CLOUD_PLAN` on Render so the Ops Dashboard reports green. Until then the verdict below stands as written: *300 in audience mode is realistic, conditional on the plan tiers.*

1 • ARCHITECTURE AS TRACED

Every attendee opens one media connection and eight realtime channels

```
Browser
└─ LiveKitRoom video={false} audio={false} RoomPage.tsx:388
    └─ LiveKit Cloud wss://honeycom-bvu9hdr5.livekit.cloud
        rooms auto-created on first join · no createRoom · no maxParticipants
        adaptiveStream: false dynacast: false autoSubscribe: true (client defaults, not
        overridden)
        └─ POST /api/livekit/token Express, one Node process on Render
            rooms.select  $\rightarrow$  [gated: room_hosts.select + participants.insert | admission upsert]  $\rightarrow$  JWT
            every attendee: canPublish true, canSubscribe true (no audience role)
        └─ Supabase Postgres 17, eu-west-1 — per client:
            participants postgres_changes · chat postgres_changes · agenda
            cursors broadcast · hands · moderation · reactions · room-coordination
            + admission (hosts)
```

Deployment	LiveKit Cloud, global mesh SFU. Plan/tier not derivable from code — UNKNOWN
Room cap	None set anywhere. <code>rooms.max_participants</code> int default 200 exists in the schema and is never read — a dead column, not a limit.
Who publishes	Nobody by default. Attendees join muted with camera off and opt in. This is the single decision that makes audience mode possible.
AI Note Taker	One audio-only room-composite egress per room (<code>meetingIntelligence.js:80</code>), duplicate-guarded. O(1) in attendees — not a bottleneck.
Video grid	LiveKit <code>GridLayout</code> paginates at a maximum of 25 tiles per page (5×5), so the DOM never holds 300 tiles.

Where 300 actually breaks

COMPONENT	CURRENT LIMIT	300+	RISK
LiveKit room	No per-room cap; plan quota unknown	LIKELY	Needs LiveKit Cloud quota confirmation for 300 participant-hours
Token generation	Measured: 350 simultaneous joins, 0 failures, 57 joins/s, p99 6.0 s	YES	None
Audio subscriptions	Every published audio subscribed; one <code><audio></code> element each; no server mixing	≤ 20 UNMUTED	300 unmuted = 300 decodes per client
Video subscriptions	<code>adaptiveStream: false</code> — every client pulls full-res from every publisher, even off-page	≤ 6–8 CAMERAS	25 cameras ≈ 35 Mbps per viewer; 300 cameras is physically impossible
Frontend grid	Paginated, 25 tiles max	YES	Placeholder array is O(300) per render — fine
Participants panel	Renders every row, no virtualization	MARGINAL	300 rows re-render on every roster event
Presence	N ² refetch — every client refetches the whole roster + profiles on every join/leave	NO	300 joins ≈ 45k change fan-outs ≈ 90,000 DB queries in the join window
Supabase Realtime	Peak connections: Free 200 · Pro 500 · plan unknown	NO ON FREE	On Free, attendee #201 cannot connect at all
Cursor sharing	30 fps × 300 recipients = 9,000 msg/s	NO	Exceeds every Supabase tier's messages-per-second quota; starves presence
Reactions / hands	Broadcast fan-out × 300	DEGRADES	An applause wave → thousands of msg/s
Chat	<code>postgres_changes</code> + a <code>__JOIN__</code> / <code>__LEAVE__</code> row per person	DEGRADES	300 system inserts × 300 subscribers, RLS-checked each
Waiting room	Host admits one attendee at a time — no bulk admit	NO	300 clicks before a scheduled conference can start
AI Note Taker	One egress per room	YES	None
Backend (Render)	Single Node process	YES	Join burst proven above
Browser	25 tiles, few decodes in audience mode	AUDIENCE MODE	Fails only if many cameras are on

3 • MAIN BOTTLENECK

Presence scales quadratically

USEPARTICIPANTS(ROOMID) — LIVEKIT_REACT_HOOKS.TSX:491

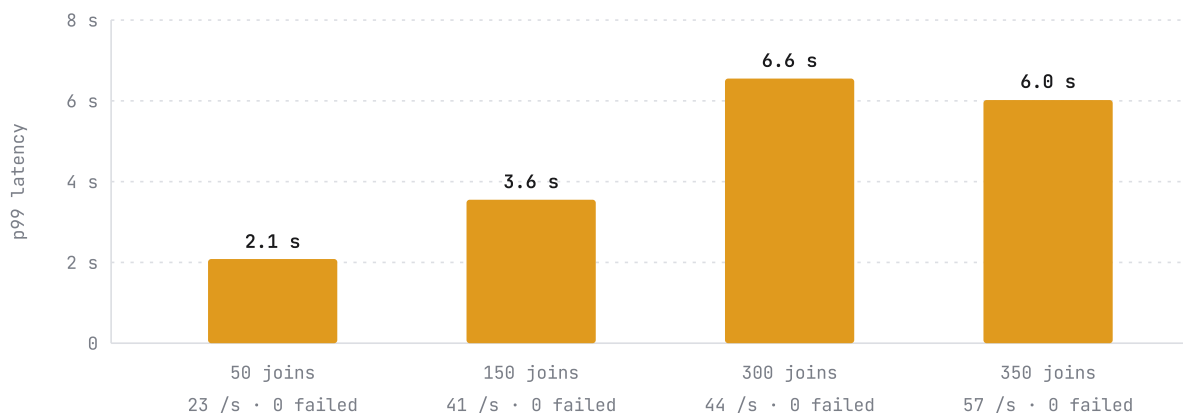
Every client subscribes to `postgres_changes` on `room_participants` and, on *any* change, re-runs a full `select *` plus a `profiles` lookup. With N attendees joining, the k -th join is refetched by all k clients already present.

```
refetches =  $\sum k$  for  $k = 1 \dots N \approx N^2 / 2$   
N = 300 → 45,150 change deliveries × 2 queries each ≈ 90,000 Postgres queries  
+ 45,150 per-subscriber RLS checks in  
Realtime
```

Invisible at ten people, catastrophic at three hundred — and it lands precisely during the mass-join burst you asked about. Either the Realtime layer throttles (stale rosters, hands, reactions) or Postgres does. It compounds with cursor sharing, which alone can emit 9,000 messages a second through the same quota.

4 • LOAD TEST

The join burst is not the problem



Fully simultaneous token requests against the production-identical backend bundle. 100 % success at every tier; the last of 300 waited 6.8 s. Latency is dominated by the round trip to Postgres in eu-west-1. Authentication and token generation are not a concern.

Not measured: the Supabase Realtime N^2 path and LiveKit media with 300 real WebRTC clients — neither is reachable from one machine. Those rows are arithmetic from the code and are marked as such.

5 • SCENARIOS

What 300 attendees can and cannot do

Mostly muted, cameras off	Media path YES . The platform fails on the presence layer, not on LiveKit.
300 cameras on	NO — and no configuration makes it yes. Every viewer would receive 300 full-res streams. Not a BeeHive limit; physics. BeeHive additionally lacks the adaptive streaming that would make even ~25 tolerable.
300 publishing audio + video	NO
AI Note Taker at 300	UNAFFECTED — one egress, guarded.

On "unlimited users per room": LiveKit Cloud's figure describes the SFU's fan-out, which was never the constraint. It means unlimited *subscribers*, not unlimited simultaneous *publishers* — the receiving browser still has to decode every published stream, so the cameras-on ceiling is set client-side on Cloud and self-hosted alike.

6 • REQUIRED BEFORE A 300+ EVENT

P0 — blockers, in the order to fix them

1

Replace polled presence with LiveKit's roster **SHIPPED**

Derive the in-room list from `useParticipants()` — already the source for the header count at `RoomPage.tsx:456` — and make `room_participants` write-only history.

ROOT CAUSE	N ² refetch on every <code>postgres_changes</code> event
FILES	<code>livekit_react_hooks.tsx</code> · <code>RoomPage.tsx</code> · <code>ParticipantsWindow.tsx</code>
COMPLEXITY	Medium
NORMAL MEETINGS	No visible change

2

Move high-frequency fan-out onto LiveKit data channels SHIPPED

Cursors, reactions and hands currently ride rate-limited Supabase broadcast. `publishData` rides the unlimited SFU mesh you already pay for, with no per-message quota.

ROOT CAUSE	9,000 msg/s cursor fan-out through a quota'd channel
FILES	RoomPage.tsx:1038-1152
COMPLEXITY	Medium
NORMAL MEETINGS	No visible change

3

Bulk admit / open-door mode for scheduled conferences SHIPPED

Add `all: true` to `/admit`, mirroring the existing `/mute` pattern at `livekit_node_backend.js:459`, plus one button in the waiting-room panel.

ROOT CAUSE	Per-request admit only
FILES	livekit_node_backend.js · AdmissionRequestsWindow.tsx
COMPLEXITY	Low
NORMAL MEETINGS	Additive

4

Confirm plan tiers

Supabase Realtime peak connections ≥ 300 (Pro or above) and the LiveKit Cloud quota for 300 participant-hours.

ROOT CAUSE	Free tier hard-stops at 200 connections
FILES	None — infrastructure
COMPLEXITY	Low
NORMAL MEETINGS	None

P1 — strongly recommended**Enable `adaptiveStream` and `dynacast`** SHIPPED

One option at `RoomPage.tsx:401`. Off-page tiles stop pulling full-res video; publishers stop encoding unused layers. Low complexity, no behaviour change in small rooms.

A real audience role **SHIPPED**

Issue `canPublish: false` tokens to attendees by default with host-granted promotion. A conference mode is technically necessary: today's uniform-publisher model is only safe because attendees *choose* not to unmute. Medium complexity.

Virtualize the participants panel **SHIPPED**

300 rows re-rendering on every roster event is the one frontend cost that grows linearly and visibly.

P2 — optimisation

Batch join/leave system chat rows

Or move them to the data channel alongside reactions.

Cap `RoomAudioRenderer` to active speakers **SHIPPED**

Bounds audio elements and decodes regardless of how many people unmute. Addressed by the 20-live-mic cap in v1.0.299.

7 · ANSWERS

The ten questions, directly

- 1. Architecture:** LiveKit Cloud mesh SFU for media; a single Express process for tokens; Supabase Realtime for presence, chat and every in-meeting signal.
- 2. Current participant limit:** none configured anywhere.
- 3. Practical capacity:** ~50 comfortable, ~100 at the edge.
- 4. Main bottleneck:** the N^2 presence refetch in `useParticipants`, compounded by Supabase Realtime quotas.
- 5. 300 with cameras/mics mostly off:** media yes; blocked by the presence layer until P0 is done.
- 6. 300 with cameras on:** no, and no configuration changes that.
- 7. 300 publishing audio and video:** no.
- 8. AI Note Taker:** does not affect scalability.
- 9. Load testing:** 350 simultaneous joins, zero failures, p99 6 s — the backend is not the constraint.
- 10. Required before a production event:** P0 items 1–4. After P0 + P1, 300 in audience mode is realistic.

Original audit was read-only against 1.0.294; the status box at the top records what shipped afterwards. Two test scripts (SSR component check, token burst) were run from the session scratchpad; nothing in the repository was modified. Values marked *unknown* require infrastructure or load-test confirmation.